

# ch1: Deep Learning: Neural Networks

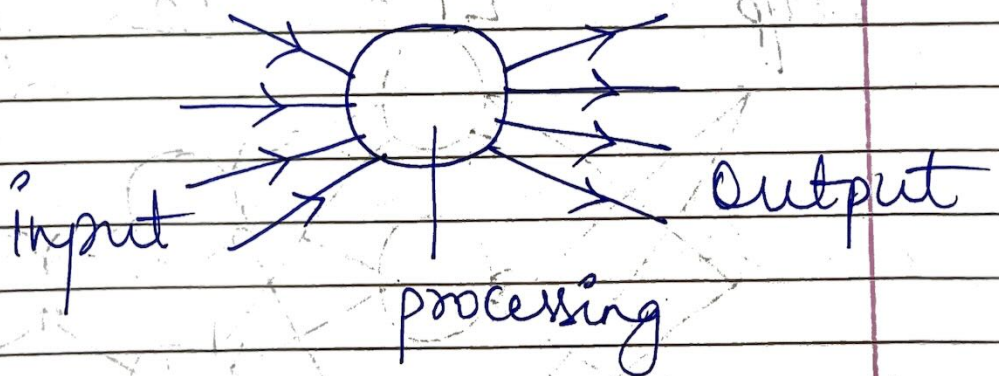
## VI: History of Neural Networks and Deep Learning

Simplest model of Neural network is Perceptron built by Rosenblatt in 1957.

It is similar to Logistic Regression.

There is a biological inspiration for perceptron. Vague understanding of brain. Neuron is a basic building block.

Neuron has biological nucleus and dendrites receives signals and passes it to other neurons through axon terminals.



Basic workflow of neuron.

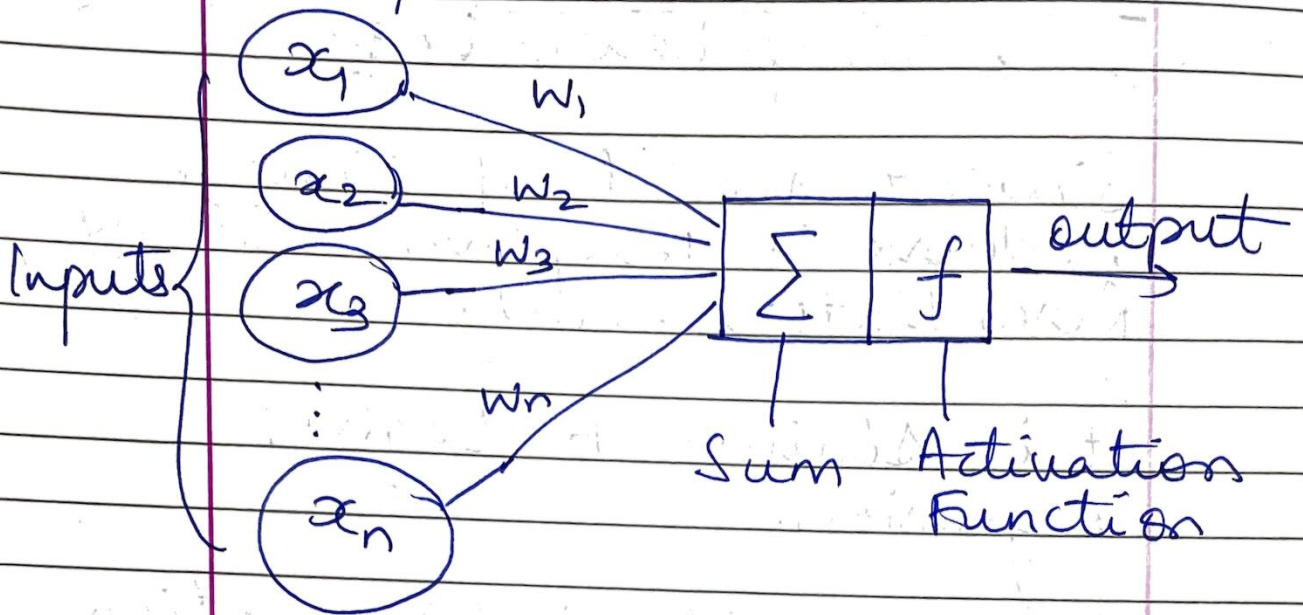




Date : \_\_\_\_\_

Page No. : \_\_\_\_\_

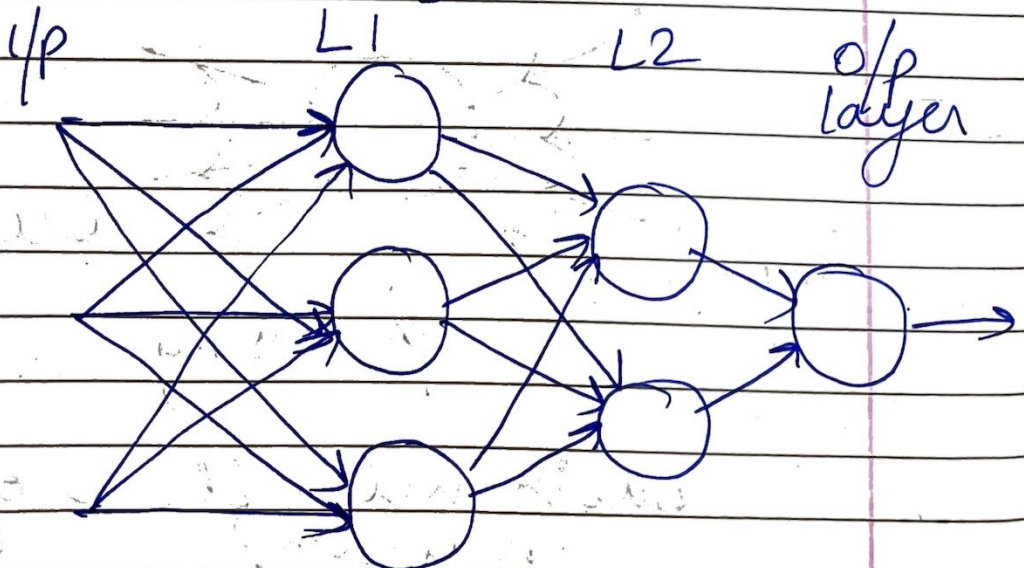
Perceptron has:



$$\Rightarrow f(w_1x_1 + w_2x_2 + w_3x_3 + \dots + w_nx_n)$$

All this is Loosely inspired from biology.

A neuron is connected to other neurons.



Network of neurons





Date :

Page No. :

From 1960's onwards, 1986 Hinton & others came up with back propagation algorithm (Chain rule of differentiation).

But we didn't have enough computational power nor the data.

So the hype that arisen then AI winter happened in 1990's which is when all the hype died.

AI was funded by US Dept of Defence.

But in 1995+ all the funding also died.

So by 1995+ SVM, RF, GBDT all this progressed.

But Jeff Hinton worked in 1986-2000 and wrote a paper on

As depth  $\uparrow$ , back propagation failed, so we required deep



neural network.

2006 Hinton come up with deep NN.

In 2012, ImageNet Competition was there (a v. hard problem)

Imagenet was conducted every year. There was error And datasets became larger & computational power increased.

Imagenet was solved by Deep NN.

Google, MS, FB, Amzn, Baidu started investing in Deep NN  
∵ they had computational power and v. large data.

Some of the voice assistants use Deep NN, Self driving Cars (Tesla, Google Waymo) & Cancer Detection System.





Date :

Page No. :

## V2: How Biological Neurons Work?

Some dendrites are thicker which means mathematically there is more weight. There is some weight with the connection input. (-ed)

A neuron is activated/fired when there is enough input, so the function associated with that neuron is called as activation function.

$$\text{So, } O = f\left(\sum_{i=1}^n w_i x_i\right)$$

where  $f$  = activation function

$w$  = weight

$x$  = input signal

$O$  = output





Date :

Page No. :

### V3: Growth of Biological Neural Networks

More neural connections, the more calories require to sustain.

### V4: Diagrammatic representations: Logistic Regression and Perceptron

We understand LR through geometric intuition and through probabilistic.

Now through Perceptron

$$\hat{y}_i = \text{sigmoid}(\sum_{j=1}^d w_j^T x_i + b)$$

Along with weights with input we add  $b$  with i/p.

$$\text{So, } \hat{y}_i = f(w^T x_i + b)$$

If  $f$  is sigmoid then the above is LR.



Training a NN means finding weights on ip-edges/nertices

$$f(x) = \begin{cases} 1 & \text{if } w^T x + b > 0 \\ 0 & \text{otherwise} \end{cases}$$

## V#5: Multi-Layered Perceptron (MLP)

Perceptron - Simple model of Simple neuron.

If function is sigmoid we get Logistic Regression.

Layers b/w ip and op layer are hidden layers.

MLP is Graphical way of representing function composition ( $f \circ g \circ h$  or  $g \circ f \circ h$ )

Since there are powerful models it can overfit easily.



## V6: Notation

$$D = \{x_i, y_i\}$$

$$x_i \in \mathbb{R}^9$$

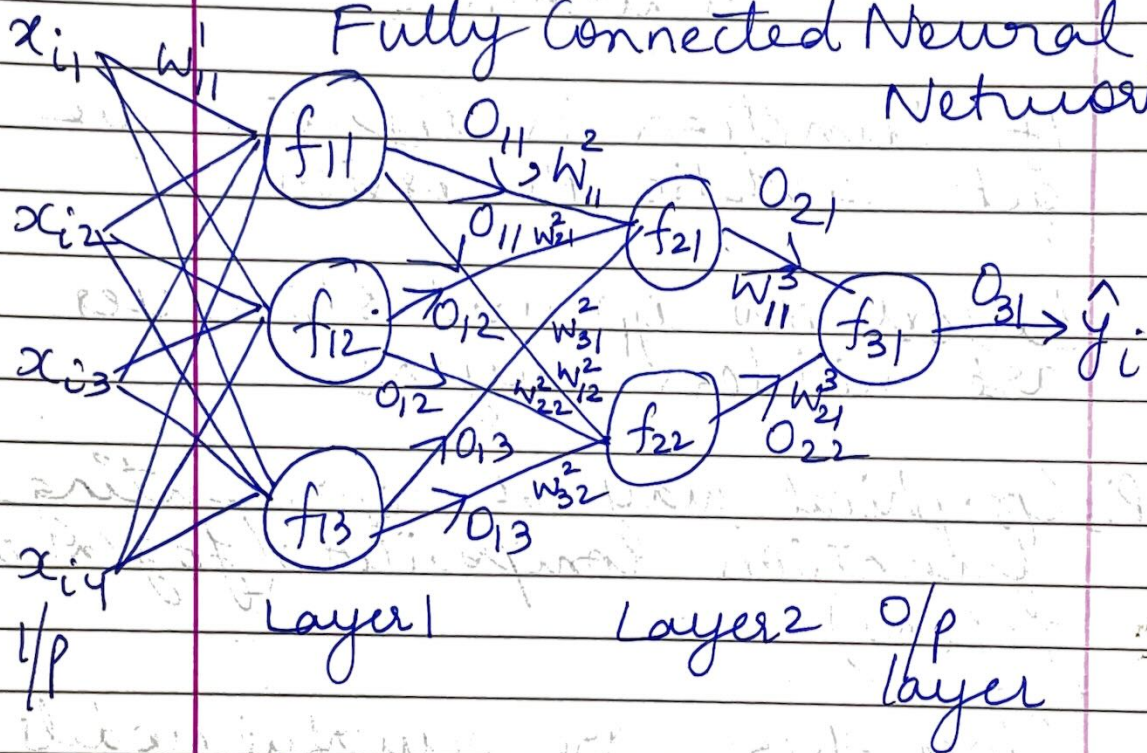
$$y_i \in \mathbb{R}$$

} Regression problem

$x_{ij}$   $i = \text{point}$

$j = \text{feature}$

Fully Connected Neural Network



$f_{ij}$ ,  $i = \text{layer \#}$   
 $j = \text{function \#}$

$O_{ij}$ ,  $i = \text{layer \#}$   
 $j = \text{neuron function \#}$

$w_{ij}^k$ ,  $k = \text{next layer \#}$  previous  
 $i = \text{neuron \# in the layer}$   
 $j = \text{neuron \# in the layer next}$





Date :

Page No. :

Weights

$$x_{i1} : \begin{bmatrix} w'_{11} & w'_{12} & w'_{13} \end{bmatrix}$$

$$x_{i2} : \begin{bmatrix} w'_{21} & w'_{22} & w'_{23} \end{bmatrix}$$

$$x_{i3} : \begin{bmatrix} w'_{31} & w'_{32} & w'_{33} \end{bmatrix}$$

$$x_{i4} : \begin{bmatrix} w'_{41} & w'_{42} & w'_{43} \end{bmatrix}_{4 \times 3}$$

Weights can be represented as matrix with some dimension on each layer.

### V7: Training a Single-neuron model

Training in NN model means to find best edge weights using  $D_{\text{train}}$

- 1) Perceptron
  - 2) Logistic Regression
  - 3) Linear Regression
- Single Neuron model for classification  
" for Regression.





Date :

Page No. :

We also optimize Linear Regression.

Linear Regression problem:-

$$\hat{y}_i = \sum_{j=1}^d w_j x_{ij}$$

$$\hat{y}_i = w^T x_i$$

Find optimal  $w$

If an i/p and o/p to a func. is same, the func. is called Identity function.

Linear Regression optimization

$$\min_{w_i} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \text{regularization}$$

$$\hat{y}_i = w^T x_i$$

Can also be written as:-

$$\min_{w_i} \sum_{i=1}^n (y_i - w^T x_i)^2 + \|w\|_2^2$$



1) Define loss func.

$$(loss) \quad L = \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \text{regularization}$$

(loss on 1 train point)  $L_i = (y_i - \hat{y}_i)^2$

2) Write the optimization problem:-

$$\min_{w_i} \sum_{i=1}^n (y_i - w^T x_i)^2 + \text{regularization}$$

can also be written as,

$$\min_{w_i} \sum_{i=1}^n (y_i - f(w^T x_i))^2 + \text{regularization}$$

If  $f = I$  <sup>identity function</sup> = Linear Regression

Sigmoid = Logistic Regression

$\mathcal{C}$  = Perceptron.

$$w^* = \underset{w}{\operatorname{argmin}} \sum_{i=1}^n (y_i - f(w^T x_i))^2 + \text{reg.}$$

3) Solve the optimization problem.  
a) Initialize of weights in SGD randomly.



Partial



Date:

Page No.:

b) Derivative of  $L = \nabla_w L$

$$= \begin{bmatrix} \frac{\partial L}{\partial w_1} \\ \frac{\partial L}{\partial w_2} \\ \vdots \\ \frac{\partial L}{\partial w_n} \end{bmatrix}$$

$$c) w_{\text{new}} = w_{\text{old}} - \underset{\substack{\text{learning} \\ \text{rate}}}{\eta} \cdot \left[ \nabla_w L \right]_{w_{\text{old}}}$$

update equation:

$$(w_i)_{\text{new}} = (w_i)_{\text{old}} - \eta \left[ \frac{\partial L}{\partial w_i} \right]_{(w_i)_{\text{old}}}$$

Do it by using iteration till ~~it~~ it converges.

In Standard GD we took all points.

⊛ But in Stochastic GD we use small batch of pts. it is called Batch SGD.

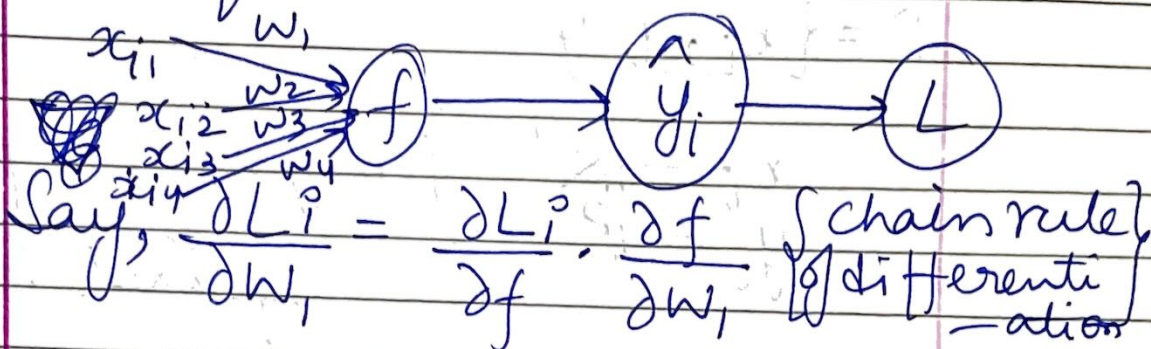




Date :

Page No. :

In Regression Loss function is Squared Loss.



$$\frac{\partial L}{\partial f} = \sum_{i=1}^n 2(y_i - f(w^T \cdot x_i))$$

$$= -2(y_i - \hat{y}_i)$$

$$\frac{\partial f}{\partial w_1} = x_i$$

After multiplying:-

$$\frac{\partial L_i}{\partial w_1} = \frac{\partial L_i}{\partial f} \cdot \frac{\partial f}{\partial w_1} = -2x_i(y_i - \hat{y}_i)$$

$$= \sum_{i=1}^n -2x_i(y_i - \hat{y}_i)$$





Date :

Page No. :

Op

## V8: MLP: Chain Rule

$$D = \{x_i, y_i\}$$

$$x_i \in \mathbb{R}^4$$

$$y_i \in \mathbb{R}$$

} Regression problem

↓  
Squared loss

In a fully connected neural network (see previous diag.) we get an o/p and loss  $L$  is applied on the o/p.

$$L(y_i, \hat{y}_i)$$

Determine  $w_{4 \times 3}^1, w_{3 \times 2}^2, w_{2 \times 1}^3 = 20$  weights

$$1) L = \sum_{i=1}^n \underbrace{(y_i - \hat{y}_i)^2}_{\text{squared loss}} + \text{regularization}$$

$$\text{reg} = \sum_{i,j,k} (w_{ij}^k)^2 \quad \text{ignore reg. right now.}$$

$$L_i = (y_i - \hat{y}_i)^2$$

$$L = \sum_{i=1}^n L_i + \text{reg.}$$



Optimization problem :-

$$\min_{w^1, w^2, w^3} L \begin{cases} \text{Squared loss} \\ \text{reg.} \end{cases}$$

$$\min_{w_{ij}^k} L$$

2) SGD

a) Initialization of variables ( $w_{ij}^k$ ) randomly.

Update Rule b)  $(w_{ij}^k)_{\text{new}} = (w_{ij}^k)_{\text{old}} - \eta \frac{\partial L}{\partial w_{ij}^k}$

learning rate      partial derivative

c) Perform update till convergence.

$$\boxed{w_{11}^3} \quad \frac{\partial L}{\partial w_{11}^3} = \frac{\partial L}{\partial o_{31}} \cdot \frac{\partial o_{31}}{\partial w_{11}^3}$$

$$\frac{\partial L}{\partial w_{21}^3} = \frac{\partial L}{\partial o_{31}} \cdot \frac{\partial o_{31}}{\partial w_{21}^3}$$



$$\boxed{W^2}$$

$$\frac{\partial L}{\partial W_{11}^2} = \frac{\partial L}{\partial O_{31}} \cdot \frac{\partial O_{31}}{\partial O_{21}} \cdot \frac{\partial O_{21}}{\partial W_{11}^2}$$

$$\frac{\partial L}{\partial W_{21}^2} = \frac{\partial L}{\partial O_{31}} \cdot \frac{\partial O_{31}}{\partial O_{21}} \cdot \frac{\partial O_{21}}{\partial W_{21}^2}$$

$$\frac{\partial L}{\partial W_{31}^2} = \frac{\partial L}{\partial O_{31}} \cdot \frac{\partial O_{31}}{\partial O_{21}} \cdot \frac{\partial O_{21}}{\partial W_{31}^2}$$

Write rest 6

$$\boxed{W^1}$$

Special case of chain rule:  $h(f(x), g(x))$

$$\frac{\partial h}{\partial x} = \frac{\partial h}{\partial f} \cdot \frac{\partial f}{\partial x} + \frac{\partial h}{\partial g} \cdot \frac{\partial g}{\partial x}$$

$$\frac{\partial L}{\partial W_{11}^1} = \frac{\partial L}{\partial O_{31}} \cdot \frac{\partial O_{31}}{\partial W_{11}^1} \cdot \frac{\partial W_{11}^1}{\partial W_{11}^1}$$

$$\frac{\partial O_{31}}{\partial W_{11}^1} = \frac{\partial O_{31}}{\partial O_{21}} \cdot \frac{\partial O_{21}}{\partial O_{11}} \cdot \frac{\partial O_{11}}{\partial W_{11}^1} + \frac{\partial O_{31}}{\partial O_{22}} \cdot \frac{\partial O_{22}}{\partial O_{11}} \cdot \frac{\partial O_{11}}{\partial W_{11}^1}$$

Apply spl. chain rule when there is bifurcation in the path.



on there is 2 output derivative in  
 the middle term we use spl. chain rule.  
~~that when a layer has 3 activation func.~~

$$\frac{\partial L}{\partial W'_{11}} = \frac{\partial L}{\partial O_{31}} \left\{ \frac{\partial O_{31}}{\partial O_{21}} \cdot \frac{\partial O_{21}}{\partial O_{11}} \cdot \frac{\partial O_{11}}{\partial W'_{11}} + \frac{\partial O_{31}}{\partial O_{22}} \cdot \frac{\partial O_{22}}{\partial O_{11}} \cdot \frac{\partial O_{11}}{\partial W'_{11}} \right\}$$

### V9: MLP: Memoization

Compute once and reuse the same derivative again.

It speeds up but uses slightly more memory.

Backpropagation = Chain Rule + Memoization

### V10: Backpropagation

$$D = \{x_i, y_i\}$$

- 1) Initialize  $W_{ij}^k$ 's randomly.
- 2) For each  $x_i$  in  $D$ 
  - ↳ pass  $x_i$  forward through the network (forward propagation)
  - ↳ Compute  $L(\hat{y}_i, y_i)$
  - ↳ Compute all the derivative using chain rule and memoization.



- ↪ update weights from end of the network to the start (backward propagation)
- 3) Repeat step 2 till convergence.  
convergence is:-  
 $(W_{ij}^k)_{\text{new}} \approx (W_{ij}^k)_{\text{old}}$

- ↪ epoch of training → Passed all points of dataset  $D$  once.
- ↪ 5 epochs → Datapoints are passed 5 times.

Pick datapoint randomly for 1 epoch.

Backpropagation :-

- 1) Initialize
- 2) epoch
  - forward propagation
  - compute loss
  - compute derivatives using chain rule and normalization
  - update weights back propagation
- 3) Convergence.

If  $f_{ij}$  are not differentiable you can't perform chain rule hence no weight updates.





Date :

Page No. :

So activation function must be differentiable. Design that may be easy and fast to differentiate.

- 1) In SGD - take 1 point at a time
- 2) Minibatch SGD - take batch of points at a time
- 3) BGD - Take all points at a time at once.

GD could be extremely time & memory consuming.  
So minibatch SGD is most popular.

Let  $D$  has 10K points, so take points in power of 2 say, 32, 64, 128, 256, etc. in a batch.

## VII: Activation Functions

From 1980's till 2000's, popular activation functions were sigmoid and tanh.

$$Z = \sum_{i=1}^n W_i x_i = W^T x$$

$$W = \begin{bmatrix} W_1 \\ W_2 \\ W_3 \\ W_4 \end{bmatrix}$$

$$x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}$$



$$f = \sigma \circ f_{ij}(z)$$

Sign  
function

$$\sigma(z) = \frac{e^z}{1+e^z} \quad \text{or} \quad \frac{1}{1+e^{-z}}$$

Compute  $\frac{d\sigma}{dz} = \sigma(z)(1-\sigma(z))$

Derivative of sigmoid is using sigmoid itself.

$$0 \leq \frac{d\sigma}{dz} < 1$$

$$a = \tanh = \frac{e^z - e^{-z}}{e^z + e^{-z}} \quad \leftarrow \text{tanh activation function.}$$

$$\frac{d \tanh}{dz} = 1 - \tanh^2(z)$$

$$-1 \leq \tanh \leq 1, \quad 0 \leq \frac{d \tanh}{dz} \leq 1$$

Note: Der. of  $\sigma$  is  $< 1$  but der. of  $\tanh$  is  $\leq 1$ .



More popular func. nowadays are ReLU, Leaky ReLU.

## V12: Vanishing Gradient problem

$$\frac{\partial L}{\partial w'_{11}} = \frac{\partial L}{\partial o_{31}} \left[ \frac{\partial o_{31}}{\partial o_{21}} \cdot \frac{\partial o_{21}}{\partial o_{11}} \cdot \frac{\partial o_{11}}{\partial w'_{11}} \right]$$

$1 \ll 10^{-3} = 0.0010 \ll \begin{matrix} 0.2 \\ 0.1 \\ 0.05 \end{matrix}$

$\begin{matrix} 0 < < 1 & 0 < < 1 & 0 < < 1 \\ \frac{\partial o_{31}}{\partial o_{21}} & \frac{\partial o_{21}}{\partial o_{11}} & \frac{\partial o_{11}}{\partial w'_{11}} \end{matrix}$

It is going into the update eq.  $\rightarrow$

$$(w'_{11})_{\text{new}} = (w'_{11})_{\text{old}} - \eta \frac{dL}{dw'_{11}}$$

$\begin{matrix} 2.5 & 1 & 0.001 \\ \text{v. small} \end{matrix}$

$$\begin{aligned} & \geq 2.5 \\ & - 0.001 \\ \hline & = 2.499 \text{ (v. close to old value)} \end{aligned}$$

$\therefore$  derivatives are less than 1  
 $\therefore$  of sigmoid activation func.  
 So, weight update becomes very hard.





Date:

Page No.:

V13: Bi

# of multiplication of derivativ  
is # of hidden layers + 1.  
This problem is called Vanishing  
gradient.

$$\text{Vanishing gradient} = \frac{dL}{dw_{ij}^k} \text{ is}$$

becoming v.v. small due to  
derivation using chain rule &  
multiplication, which is  $\leq 1$ ,  
using sigmoid & tanh activation  
function.

Exploding gradient occurs in  
Recurrent NN (RNN), which is  
exact opp. of Vanishing gradient.  
It becomes too large (the derivati  
So you will never converge (we)  
and the derivative keeps changing  
Also weight becomes too variable  
in nature.

Value is either very large +ve  
or v. large negative.



## V13: Bias-Variance Tradeoff

- 1) As the # of layers  $\uparrow$ , more weights would be there. There would be higher chance of overfitting. Hence High Variance
- 2) 1-layer = Logistic Regression. Very few layer means underfitting which means high bias, hence underfit

MLP have a problem of overfitting. MLP is an optimization problem, and whenever we have overfitting problem in optimizations, we add regularization.

$$L = \sum_{i=1}^n \text{loss}(y_i, \hat{y}_i) + \sum_{j,k} (w_{ij}^k)^2$$

We can either use  $L_2$  regularization or  $L_1$  regularization.

$$L = \sum_{i=1}^n \text{loss}_i + \lambda L_2 \text{ regularization on weights}$$

Larger  $\lambda$  will have lesser overfit problem.





Date :

Page No. :

Ch 2: D

If you have  $L_1$  regularization instead of  $L_2$ , they have a problem of sparsity.

Hyperparameters:-

- 1)  $\lambda$  of  $L_2$  regularizer
- 2) # of layers

If # of layers is  $T$ , variance  $T$ .